



UNIVERSIDAD ARGENTINA DE LA EMPRESA
Facultad de Ingeniería y Ciencias Exactas - FAIN

Materia: PROCESO DE DESARROLLO DE SOFTWARE

Trabajo Práctico Obligatorio – Proceso de Desarrollo de Software

Fase 1: Análisis del problema y diseño inicial

03-06-2026 - Buenos Aires, Argentina

Integrantes del grupo:

Cortés, Ignacio 1203485

Etchart, Facundo 1201391

Rubachin, Sofía 1196599

Profesor:

Ing. Leon Maria Angela

Índice

Índice.....	1
1. Descripción general de la problemática.....	2
2. Objetivos del sistema.....	2
Objetivo general.....	2
Objetivos específicos.....	2
3. Alcance funcional.....	2
4. Requisitos funcionales.....	3
5. Requisitos no funcionales básicos.....	3
6. Diagrama de clases.....	4
7. Identificación de actores o usuarios.....	5
8. Casos de uso principales.....	5
9. Diagrama de casos de uso.....	5
10. Primer listado de clases candidatas.....	7
11. Repositorio Git creado y compartido con la docente.....	7
Justificación breve de diseño.....	7
Anexo.....	8
• Diagrama de clases en PlantUML.....	8
• Diagrama de casos de uso en PlantUML.....	12

Sistema de Gestión Operativa para un Barrio Cerrado

1. Descripción general de la problemática

Una empresa que administra barrios cerrados necesita un sistema que le permita organizar mejor las tareas operativas, administrativas y de seguridad.

Actualmente, muchas actividades se manejan con planillas, registros manuales o aplicaciones separadas. Esto genera pérdida de información, demoras en la resolución de reclamos y poca claridad sobre el estado de cada solicitud.

El sistema busca centralizar la gestión del barrio, permitiendo registrar residentes, viviendas, visitantes, proveedores, reclamos, tareas de mantenimiento e incidentes de seguridad. De esta forma, la administración podrá tener un mejor seguimiento de las actividades y los residentes podrán recibir respuestas más claras y rápidas.

2. Objetivos del sistema

Objetivo general

Desarrollar un sistema que permita centralizar la gestión operativa de un barrio cerrado, mejorando el seguimiento de reclamos, tareas, accesos y comunicaciones entre residentes y administradores.

Objetivos específicos

- Registrar barrios cerrados, residentes y viviendas.
- Registrar visitantes y proveedores.
- Gestionar reclamos realizados por residentes.
- Gestionar tareas de mantenimiento.
- Registrar incidentes de seguridad.
- Cambiar el estado de una solicitud según su avance.
- Notificar novedades a residentes o administradores.
- Mejorar la organización y trazabilidad de las actividades del barrio.

3. Alcance funcional

El sistema permitirá administrar las funciones principales de un barrio cerrado.

Incluye:

- Gestión de barrios cerrados
- Gestión de residentes.

- Gestión de viviendas.
- Gestión de visitantes.
- Gestión de personal autorizado.
- Control básico de accesos.
- Registro y seguimiento de reclamos.
- Registro de tareas de mantenimiento.
- Registro de incidentes de seguridad.
- Notificaciones a usuarios.

No incluye en esta primera etapa:

- Pago de expensas.
- Cámaras de seguridad.
- Reconocimiento facial.
- Apertura automática de barreras.
- Integración con sistemas externos.

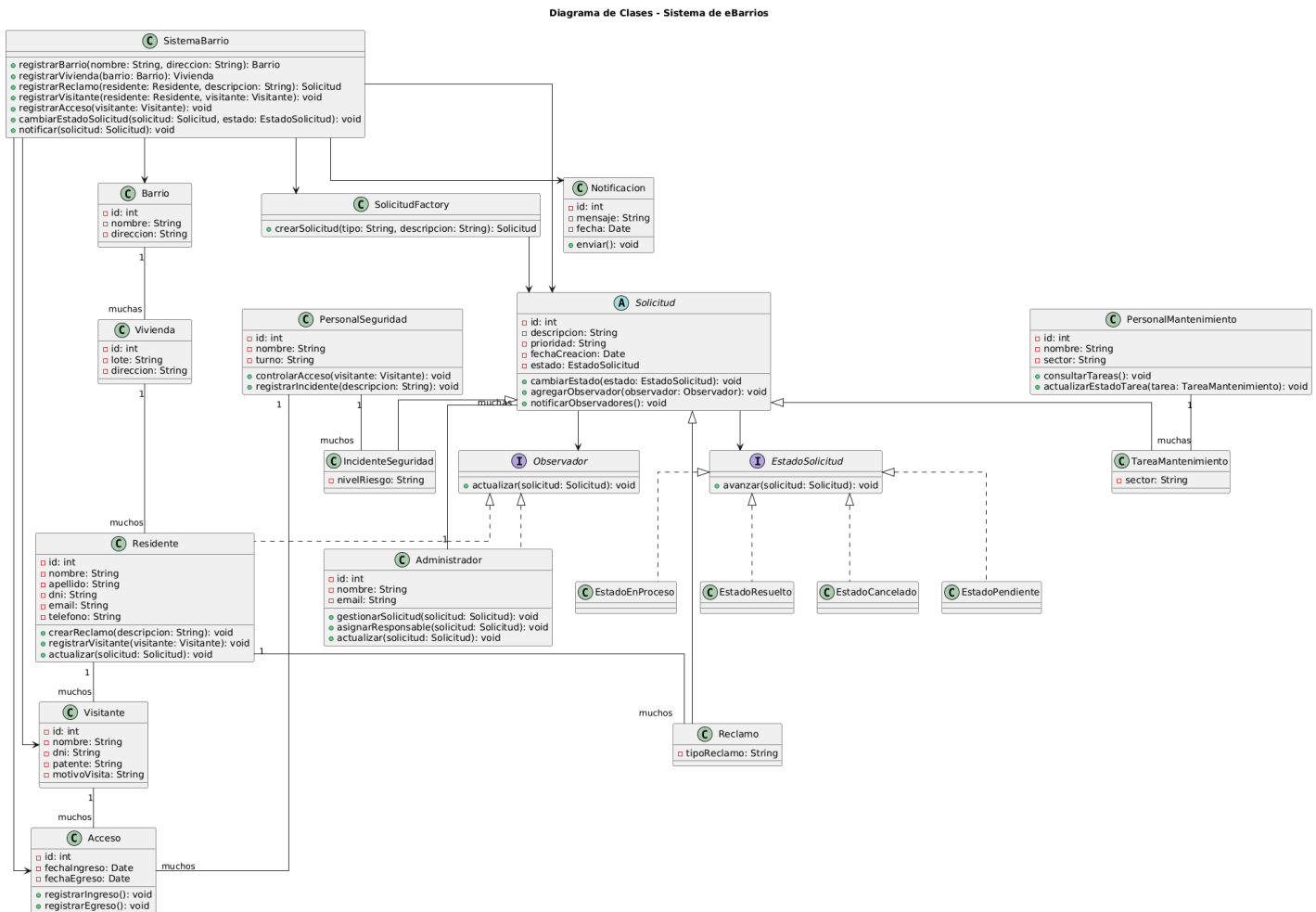
4. Requisitos funcionales

- El sistema debe permitir registrar barrios cerrados.
- El sistema debe permitir registrar residentes.
- El sistema debe permitir registrar viviendas.
- El sistema debe permitir asociar viviendas a un barrio.
- El sistema debe permitir asociar residentes a una vivienda.
- El sistema debe permitir registrar visitantes.
- El sistema debe permitir registrar proveedores.
- El sistema debe permitir registrar reclamos.
- El sistema debe permitir registrar tareas de mantenimiento.
- El sistema debe permitir registrar incidentes de seguridad.
- El sistema debe permitir cambiar el estado de una solicitud.
- El sistema debe permitir consultar el estado de una solicitud.
- El sistema debe permitir notificar novedades a residentes o administradores.
- El sistema debe permitir registrar ingresos y egresos de visitantes o proveedores.

5. Requisitos no funcionales básicos

- **Usabilidad:** el sistema debe ser fácil de usar para usuarios sin conocimientos técnicos.
- **Seguridad:** la información de residentes y accesos debe estar protegida. Además, sólo los usuarios autorizados podrán acceder a funciones administrativas.
- **Disponibilidad:** el sistema debe poder utilizarse cuando sea necesario.
- **Mantenibilidad:** el sistema debe estar organizado para poder modificarse fácilmente.
- **Escalabilidad:** debe permitir agregar nuevas funcionalidades en el futuro sin requerir modificaciones significativas sobre las existentes.
- **Trazabilidad:** debe guardar el historial de solicitudes, accesos, reclamos, tareas e incidentes.

6. Diagrama de clases



El diseño propuesto aplica principios GRASP y SOLID mediante una separación clara de responsabilidades. Cada clase representa un concepto específico del sistema y se evita concentrar toda la lógica en una única clase.

Se aplica **Information Expert** porque cada clase administra los datos que le corresponden, como Solicitud con su estado y prioridad o Acceso con los ingresos y egresos. También se aplica **Creator** mediante SolicitudFactory, encargada de crear los distintos tipos de solicitudes. La clase SistemaBarrio funciona como **Controller** y **Facade**, ya que coordina las operaciones principales del sistema y ofrece una interfaz simple para registrar reclamos, accesos y cambios de estado.

En cuanto a SOLID, el diseño respeta la responsabilidad única, ya que cada clase tiene una función concreta. También permite extender el sistema agregando nuevos tipos de solicitudes o nuevos estados sin modificar toda la estructura. Las clases Reclamo, TareaMantenimiento e IncidenteSeguridad pueden utilizarse como Solicitud, cumpliendo sustitución de Liskov. Además, se utiliza la interfaz Observador para notificar cambios, reduciendo el acoplamiento entre clases.

Los patrones aplicados son **Factory Method**, para crear solicitudes; **Observer**, para notificar cambios de estado; **State**, para representar los estados de una solicitud; y **Facade**, mediante la clase SistemaBarrio, que simplifica el acceso a las operaciones principales.

7. Identificación de actores o usuarios

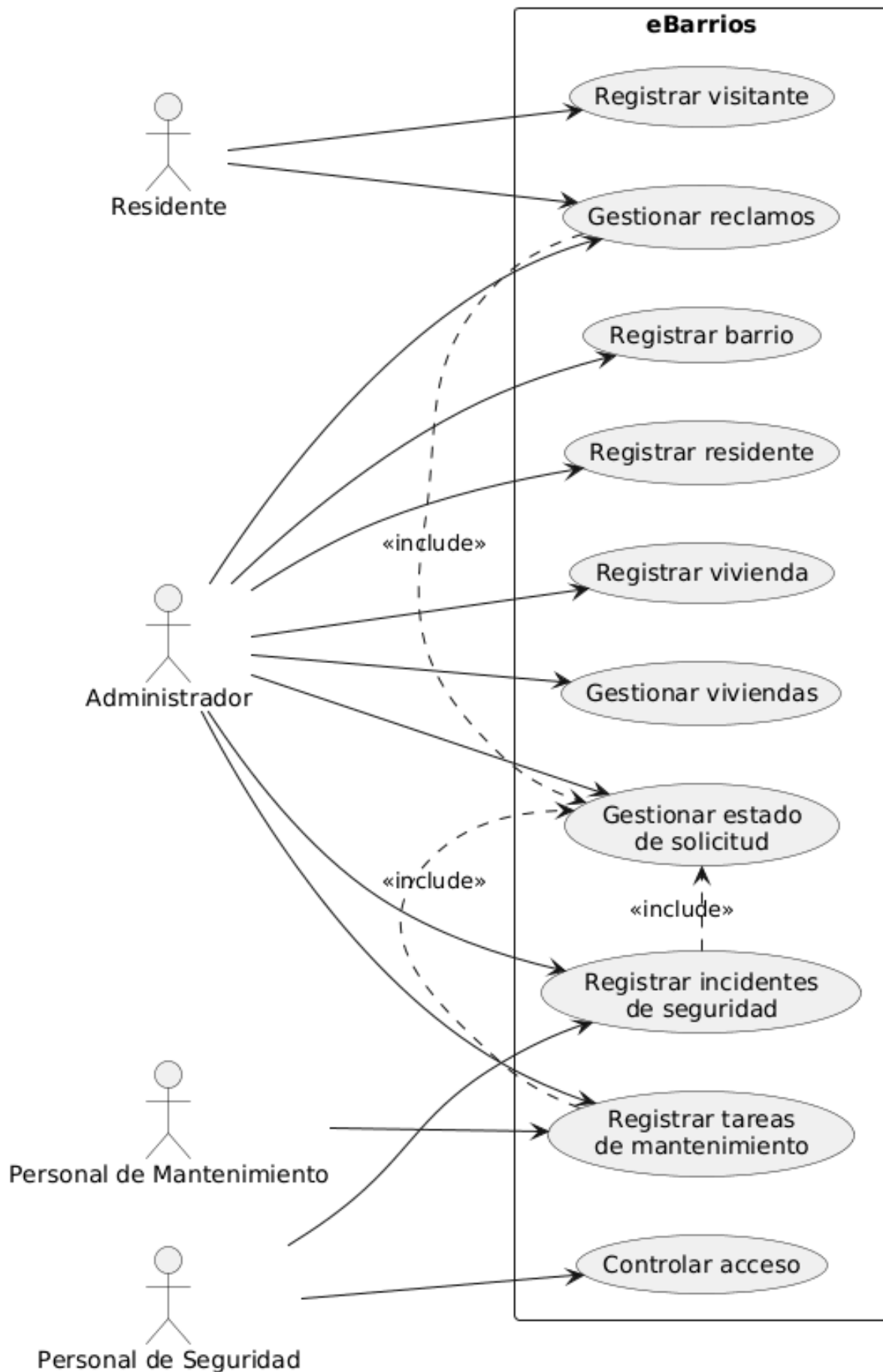
- **Residente:** persona que vive en el barrio. Puede registrar visitantes, realizar reclamos y consultar el estado de sus solicitudes.
- **Administrador:** persona encargada de gestionar residentes, viviendas, reclamos, tareas y notificaciones.
- **Personal de seguridad:** controla ingresos y egresos de visitantes o proveedores.
- **Personal de mantenimiento:** atiende tareas asignadas.
- **Visitante:** persona autorizada por un residente para ingresar al barrio.

8. Casos de uso principales

- Registrar barrio.
- Registrar residente.
- Registrar vivienda.
- Registrar visitante.
- Registrar proveedor.
- Controlar acceso.
- Gestionar viviendas.
- Gestionar reclamos.
- Registrar tareas de mantenimiento.
- Registrar incidentes de seguridad.
- Gestionar estado de solicitud.

9. Diagrama de casos de uso

Diagrama de Casos de Uso - eBarrios



10. Primer listado de clases candidatas

- Barrio
- Residente
- Administrador
- Vivienda
- Visitante
- Solicitud
- Reclamo
- TareaMantenimiento
- IncidenteSeguridad
- Notificacion
- Acceso
- PersonalSeguridad
- PersonalMantenimiento

11. Repositorio Git creado y compartido con la docente

Se creó un repositorio en GitHub para organizar el proyecto y registrar los avances del trabajo práctico.

El repositorio incluirá las futuras implementaciones del sistema.

Repositorio GitHub: <https://github.com/sofiruba/eBarrio>

Justificación breve de diseño

El diseño inicial busca mantener una estructura simple y ordenada. Las clases representan los elementos principales del barrio cerrado, como residentes, viviendas, visitantes, proveedores, solicitudes y accesos.

Además, se separan las responsabilidades principales: los residentes realizan reclamos o registran visitantes, los administradores gestionan solicitudes y el personal de seguridad controla accesos. Esto permite que el sistema sea más fácil de entender, modificar y ampliar en futuras etapas.

Más adelante se podrán aplicar patrones de diseño como **Factory Method** para crear distintos tipos de solicitudes, **Observer** para notificar cambios de estado y **State** para manejar los estados de reclamos, tareas o incidentes.

Anexo

- Diagrama de clases en PlantUML

```
@startuml
skinparam linetype ortho
title Diagrama de Clases - Sistema de eBarrios

class SistemaBarrio {
+ registrarBarrio(nombre: String, direccion: String): Barrio
+ registrarVivienda(barrio: Barrio): Vivienda
+ registrarReclamo(residente: Residente, descripcion: String): Solicitud
+ registrarVisitante(residente: Residente, visitante: Visitante): void
+ registrarAcceso(visitante: Visitante): void
+ cambiarEstadoSolicitud(solicitud: Solicitud, estado: EstadoSolicitud): void
+ notificar(solicitud: Solicitud): void
}

class Barrio {
- id: int
- nombre: String
- direccion: String
}

class Residente {
- id: int
- nombre: String
- apellido: String
- dni: String
- email: String
- telefono: String

+ crearReclamo(descripcion: String): void
+ registrarVisitante(visitante: Visitante): void
+ actualizar(solicitud: Solicitud): void
}

class Administrador {
- id: int
- nombre: String
- email: String

+ gestionarSolicitud(solicitud: Solicitud): void
+ asignarResponsable(solicitud: Solicitud): void
+ actualizar(solicitud: Solicitud): void
}
```

```

class PersonalSeguridad {
- id: int
- nombre: String
- turno: String

+ controlarAcceso(visitante: Visitante): void
+ registrarIncidente(descripcion: String): void
}

class PersonalMantenimiento {
- id: int
- nombre: String
- sector: String

+ consultarTareas(): void
+ actualizarEstadoTarea(tarea: TareaMantenimiento): void
}

class Vivienda {
- id: int
- lote: String
- direccion: String
}

class Visitante {
- id: int
- nombre: String
- dni: String
- patente: String
- motivoVisita: String
}

class Acceso {
- id: int
- fechaIngreso: Date
- fechaEgreso: Date

+ registrarIngreso(): void
+ registrarEgreso(): void
}

abstract class Solicitud {
- id: int
- descripcion: String
- prioridad: String
- fechaCreacion: Date
- estado: EstadoSolicitud

```

```

+ cambiarEstado(estado: EstadoSolicitud): void
+ agregarObservador(observador: Observador): void
+ notificarObservadores(): void
}

class Reclamo {
- tipoReclamo: String
}

class TareaMantenimiento {
- sector: String
}

class IncidenteSeguridad {
- nivelRiesgo: String
}

interface EstadoSolicitud {
+ avanzar(solicitud: Solicitud): void
}

class EstadoPendiente
class EstadoEnProceso
class EstadoResuelto
class EstadoCancelado

interface Observador {
+ actualizar(solicitud: Solicitud): void
}

class SolicitudFactory {
+ crearSolicitud(tipo: String, descripcion: String): Solicitud
}

class Notificacion {
- id: int
- mensaje: String
- fecha: Date

+ enviar(): void
}

'=====
' Relaciones de dominio
'=====

```

Barrio "1" -- "muchas" Vivienda

Vivienda "1" -- "muchos" Residente

Residente "1" -- "muchos" Visitante

Residente "1" -- "muchos" Reclamo

Visitante "1" -- "muchos" Acceso

'=====

' Herencia

'=====

Solicitud <|-- Reclamo

Solicitud <|-- TareaMantenimiento

Solicitud <|-- IncidenteSeguridad

'=====

' State

'=====

EstadoSolicitud <|.. EstadoPendiente

EstadoSolicitud <|.. EstadoEnProceso

EstadoSolicitud <|.. EstadoResuelto

EstadoSolicitud <|.. EstadoCancelado

Solicitud --> EstadoSolicitud

'=====

' Observer

'=====

Observador <|.. Residente

Observador <|.. Administrador

Solicitud --> Observador

'=====

' Factory

'=====

SolicitudFactory --> Solicitud

'=====

' Facade / Controller

'=====

SistemaBarrio --> Barrio

SistemaBarrio --> SolicitudFactory

SistemaBarrio --> Solicitud
SistemaBarrio --> Visitante
SistemaBarrio --> Acceso
SistemaBarrio --> Notificacion

'=====

' Gestión

'=====

Administrador "1" -- "muchas" Solicitud

PersonalSeguridad "1" -- "muchos" Acceso
PersonalSeguridad "1" -- "muchos" IncidenteSeguridad

PersonalMantenimiento "1" -- "muchas" TareaMantenimiento
@enduml

- **Diagrama de casos de uso en PlantUML**

@startuml

title Diagrama de Casos de Uso - eBarrios

left to right direction

actor Residente

actor Administrador

actor "Personal de Seguridad" as Seguridad

actor "Personal de Mantenimiento" as Mantenimiento

rectangle "eBarrios" {

 usecase "Registrar barrio" as UC1

 usecase "Registrar residente" as UC2

 usecase "Registrar vivienda" as UC3

 usecase "Registrar visitante" as UC4

 usecase "Gestionar viviendas" as UC5

 usecase "Gestionar reclamos" as UC6

 usecase "Registrar tareas\nde mantenimiento" as UC7

 usecase "Registrar incidentes\nde seguridad" as UC8

```

    usecase "Controlar acceso" as UC9

    usecase "Gestionar estado\nde solicitud" as UC10
}

```

```

'=====
' Residente
'=====

```

```

Residente --> UC4
Residente --> UC6

```

```

'=====
' Administrador
'=====

```

```

Administrador --> UC1
Administrador --> UC2
Administrador --> UC3
Administrador --> UC5
Administrador --> UC6
Administrador --> UC7
Administrador --> UC8
Administrador --> UC10

```

```

'=====
' Seguridad
'=====

```

```

Seguridad --> UC9
Seguridad --> UC8

```

```

'=====
' Mantenimiento
'=====

```

```

Mantenimiento --> UC7

```

```

'=====
' Includes
'=====

```

```

UC6 .> UC10 : <<include>>
UC7 .> UC10 : <<include>>
UC8 .> UC10 : <<include>>
@enduml

```